

d76 - Google Play Store App Analytics (start)

January 28, 2026

1 Introduction

In this notebook, we will do a comprehensive analysis of the Android app market by comparing thousands of apps in the Google Play store.

2 About the Dataset of Google Play Store Apps & Reviews

Data Source: App and review data was scraped from the Google Play Store by Lavanya Gupta in 2018. Original files listed [here](#).

3 Import Statements

```
[1]: import pandas as pd
```

4 Notebook Presentation

```
[2]: # Show numeric output in decimal format e.g., 2.15
pd.options.display.float_format = '{:,.2f}'.format
```

5 Read the Dataset

```
[3]: df_apps = pd.read_csv('apps.csv')
```

6 Data Cleaning

Challenge: How many rows and columns does `df_apps` have? What are the column names? Look at a random sample of 5 different rows with `.sample()`.

```
[4]: #df_apps
# df_apps.tail()
# df_apps.describe()
# df_apps.shape
df_apps.columns
# df_apps.isna().values.any():
# df_apps.sample(5)
```

```
[4]: Index(['App', 'Category', 'Rating', 'Reviews', 'Size_MBs', 'Installs', 'Type',
          'Price', 'Content_Rating', 'Genres', 'Last_Updated', 'Android_Ver'],
          dtype='object')
```

```
[5]: df_apps.sample(5)
```

```
[5]:
```

	App	Category	Rating	Reviews	\
2993	Tinnitus Alleviator	MEDICAL	3.10	82	
4574	Dr. Panda & Toto's Treehouse	FAMILY	4.40	3396	
5127	EL NORTE	NEWS_AND_MAGAZINES	3.40	2436	
4829	DataCamp - Learn R, Python & SQL	FAMILY	4.40	944	
10468	Photo Editor Collage Maker Pro	PHOTOGRAPHY	4.50	1519671	

	Size_MBs	Installs	Type	Price	Content_Rating	Genres	\
2993	42.00	5,000	Free	0	Everyone	Medical	
4574	9.50	50,000	Paid	\$3.99	Everyone	Casual;Pretend Play	
5127	6.30	100,000	Free	0	Everyone	News & Magazines	
4829	12.00	100,000	Free	0	Everyone	Education	
10468	6.90	100,000,000	Free	0	Everyone	Photography	

	Last_Updated	Android_Ver
2993	December 23, 2017	4.1 and up
4574	December 9, 2014	4.0 and up
5127	June 26, 2018	Varies with device
4829	August 1, 2018	4.1 and up
10468	February 1, 2018	Varies with device

```
[6]: df_apps.isna().sum()
```

```
[6]: App          0
Category        0
Rating         1474
Reviews         0
Size_MBs        0
Installs        0
Type            1
Price           0
Content_Rating  0
Genres          0
Last_Updated    0
Android_Ver     2
dtype: int64
```

```
[7]: df_apps[df_apps.Rating.isna()]
```

```
[7]:
```

	App	Category	\
0	Ak Parti Yardım Toplama	SOCIAL	
1	Ain Arabic Kids Alif Ba ta	FAMILY	

2	Popsicle Launcher for Android P 9.0 launcher	PERSONALIZATION
3	Command & Conquer: Rivals	FAMILY
4	CX Network	BUSINESS
...	...	...
5840	Em Fuga Brasil	FAMILY
5862	Voice Tables - no internet	PARENTING
6141	Young Speeches	LIBRARIES_AND_DEMO
7035	SD card backup	TOOLS
7175	Android TV Remote Service	TOOLS

	Rating	Reviews	Size_MBs	Installs	Type	Price	Content_Rating \
0	NaN	0	8.70	0	Paid	\$13.99	Teen
1	NaN	0	33.00	0	Paid	\$2.99	Everyone
2	NaN	0	5.50	0	Paid	\$1.49	Everyone
3	NaN	0	19.00	0	NaN	0	Everyone 10+
4	NaN	0	10.00	0	Free	0	Everyone
...	...	...	...	...	...	...	...
5840	NaN	1317	60.00	100,000	Free	0	Everyone
5862	NaN	970	71.00	100,000	Free	0	Everyone
6141	NaN	2221	2.40	500,000	Free	0	Everyone
7035	NaN	142	3.40	1,000,000	Free	0	Everyone
7175	NaN	1	3.70	1,000,000	Free	0	Everyone

	Genres	Last_Updated	Android_Ver
0	Social	July 28, 2017	4.1 and up
1	Education	April 15, 2016	3.0 and up
2	Personalization	July 11, 2018	4.2 and up
3	Strategy	June 28, 2018	Varies with device
4	Business	August 6, 2018	4.1 and up
...	...	...	...
5840	Simulation	April 22, 2018	4.1 and up
5862	Parenting	May 28, 2018	4.0.3 and up
6141	Libraries & Demo	January 8, 2017	2.3 and up
7035	Tools	March 27, 2017	Varies with device
7175	Tools	January 17, 2018	7.0 and up

[1474 rows x 12 columns]

6.0.1 Drop Unused Columns

Challenge: Remove the columns called `Last_Updated` and `Android_Version` from the `DataFrame`. We will not use these columns.

```
[8]: df_apps.drop(['Last_Updated', 'Android_Ver'], axis = 1, inplace = True)
```

```
[9]: df_apps
```

```
[9]:
```

	App	Category	Rating \
0	Ak Parti Yardım Toplama	SOCIAL	NaN
1	Ain Arabic Kids Alif Ba ta	FAMILY	NaN
2	Popsicle Launcher for Android P 9.0 launcher	PERSONALIZATION	NaN
3	Command & Conquer: Rivals	FAMILY	NaN
4	CX Network	BUSINESS	NaN
...	...	...	...
10836	Subway Surfers	GAME	4.50
10837	Subway Surfers	GAME	4.50
10838	Subway Surfers	GAME	4.50
10839	Subway Surfers	GAME	4.50
10840	Subway Surfers	GAME	4.50

	Reviews	Size_MBs	Installs	Type	Price	Content_Rating \
0	0	8.70	0	Paid	\$13.99	Teen
1	0	33.00	0	Paid	\$2.99	Everyone
2	0	5.50	0	Paid	\$1.49	Everyone
3	0	19.00	0	NaN	0	Everyone 10+
4	0	10.00	0	Free	0	Everyone
...	...	...	...	...	...	...
10836	27723193	76.00	1,000,000,000	Free	0	Everyone 10+
10837	27724094	76.00	1,000,000,000	Free	0	Everyone 10+
10838	27725352	76.00	1,000,000,000	Free	0	Everyone 10+
10839	27725352	76.00	1,000,000,000	Free	0	Everyone 10+
10840	27711703	76.00	1,000,000,000	Free	0	Everyone 10+

	Genres
0	Social
1	Education
2	Personalization
3	Strategy
4	Business
...	...
10836	Arcade
10837	Arcade
10838	Arcade
10839	Arcade
10840	Arcade

[10841 rows x 10 columns]

6.0.2 Find and Remove NaN values in Ratings

Challenge: How many rows have a NaN value (not-a-number) in the Ratings column? Create DataFrame called `df_apps_clean` that does not include these rows.

```
[10]: df_apps.isna().sum()
```

```
[10]: App          0
      Category      0
      Rating      1474
      Reviews       0
      Size_MBs     0
      Installs     0
      Type         1
      Price        0
      Content_Rating 0
      Genres       0
      dtype: int64
```

```
[11]: print(f"There are {df_apps.Rating.isna().sum()} NaN values in the Ratings_
      ↪column")
```

There are 1474 NaN values in the Ratings column

```
[5]: df_apps_clean = df_apps.dropna()
```

```
[6]: df_apps.shape
```

```
[6]: (10841, 12)
```

```
[7]: df_apps_clean.shape
```

```
[7]: (9365, 12)
```

```
[8]: df_apps_clean
```

```
[8]:
```

	App	Category	Rating	\
21	KBA-EZ Health Guide	MEDICAL	5.00	
28	Ra Ga Ba	GAME	5.00	
47	Mu.F.O.	GAME	5.00	
82	Brick Breaker BR	GAME	5.00	
99	Anatomy & Physiology Vocabulary Exam Review App	MEDICAL	5.00	
...	...	...	...	
10836	Subway Surfers	GAME	4.50	
10837	Subway Surfers	GAME	4.50	
10838	Subway Surfers	GAME	4.50	
10839	Subway Surfers	GAME	4.50	
10840	Subway Surfers	GAME	4.50	

	Reviews	Size_MBs	Installs	Type	Price	Content_Rating	Genres	\
21	4	25.00	1	Free	0	Everyone	Medical	
28	2	20.00	1	Paid	\$1.49	Everyone	Arcade	
47	2	16.00	1	Paid	\$0.99	Everyone	Arcade	
82	7	19.00	5	Free	0	Everyone	Arcade	
99	1	4.60	5	Free	0	Everyone	Medical	

```

...      ...      ...
10836  27723193      76.00  1,000,000,000  Free      0  Everyone 10+  Arcade
10837  27724094      76.00  1,000,000,000  Free      0  Everyone 10+  Arcade
10838  27725352      76.00  1,000,000,000  Free      0  Everyone 10+  Arcade
10839  27725352      76.00  1,000,000,000  Free      0  Everyone 10+  Arcade
10840  27711703      76.00  1,000,000,000  Free      0  Everyone 10+  Arcade

```

```

      Last_Updated  Android_Ver
21      August 2, 2018  4.0.3 and up
28      February 8, 2017  2.3 and up
47      March 3, 2017    2.3 and up
82      July 23, 2018    4.1 and up
99      August 2, 2018   4.0 and up

```

```

...      ...
10836      July 12, 2018  4.1 and up
10837      July 12, 2018  4.1 and up
10838      July 12, 2018  4.1 and up
10839      July 12, 2018  4.1 and up
10840      July 12, 2018  4.1 and up

```

[9365 rows x 12 columns]

6.0.3 Find and Remove Duplicates

Challenge: Are there any duplicates in data? Check for duplicates using the `.duplicated()` function. How many entries can you find for the “Instagram” app? Use `.drop_duplicates()` to remove any duplicates from `df_apps_clean`.

```
[15]: df_apps_clean.duplicated()
```

```

[15]: 21      False
      28      False
      47      False
      82      False
      99      False
      ...
10836  False
10837  False
10838  False
10839   True
10840  False
Length: 9365, dtype: bool

```

```
[17]: df_apps_clean[df_apps_clean.duplicated()]
```

```

[17]:
      App      Category  Rating \
946      420 BZ Budeze Delivery  MEDICAL  5.00
1133      MouseMingle      DATING  2.70

```

1196	Cardiac diagnosis (heart rate, arrhythmia)	MEDICAL	4.40
1231	Sway Medical	MEDICAL	5.00
1247	Chat Kids - Chat Room For Kids	DATING	4.70
...	...	...	...
10802	Skype - free IM & video calls	COMMUNICATION	4.10
10809	Instagram	SOCIAL	4.50
10826	Google Drive	PRODUCTIVITY	4.40
10832	Google News	NEWS_AND_MAGAZINES	3.90
10839	Subway Surfers	GAME	4.50

	Reviews	Size_MBs	Installs	Type	Price	Content_Rating	\
946	2	11.00	100	Free	0	Mature 17+	
1133	3	3.90	100	Free	0	Mature 17+	
1196	8	6.50	100	Paid	\$12.99	Everyone	
1231	3	22.00	100	Free	0	Everyone	
1247	6	4.90	100	Free	0	Mature 17+	
...	...	...	...	...	...	...	...
10802	10484169	3.50	1,000,000,000	Free	0	Everyone	
10809	66577313	5.30	1,000,000,000	Free	0	Teen	
10826	2731211	4.00	1,000,000,000	Free	0	Everyone	
10832	877635	13.00	1,000,000,000	Free	0	Teen	
10839	27725352	76.00	1,000,000,000	Free	0	Everyone 10+	

	Genres	Last_Updated	Android_Ver
946	Medical	June 6, 2018	4.1 and up
1133	Dating	July 17, 2018	4.4 and up
1196	Medical	July 25, 2018	3.0 and up
1231	Medical	July 25, 2018	5.0 and up
1247	Dating	July 24, 2018	4.0.3 and up
...	...	...	...
10802	Communication	August 3, 2018	Varies with device
10809	Social	July 31, 2018	Varies with device
10826	Productivity	August 6, 2018	Varies with device
10832	News & Magazines	August 1, 2018	4.4 and up
10839	Arcade	July 12, 2018	4.1 and up

[474 rows x 12 columns]

```
[17]: duplicates_instagram = df_apps_clean[df_apps_clean['App'] == 'Instagram']
```

```
[18]: duplicates_instagram
```

```
[18]:
```

	App	Category	Rating	Reviews	Size_MBs	Installs	Type	\
10806	Instagram	SOCIAL	4.50	66577313	5.30	1,000,000,000	Free	
10808	Instagram	SOCIAL	4.50	66577446	5.30	1,000,000,000	Free	
10809	Instagram	SOCIAL	4.50	66577313	5.30	1,000,000,000	Free	
10810	Instagram	SOCIAL	4.50	66509917	5.30	1,000,000,000	Free	

	Price	Content_Rating	Genres
10806	0	Teen	Social
10808	0	Teen	Social
10809	0	Teen	Social
10810	0	Teen	Social

```
[19]: # df_apps_clean = df_apps_clean.drop_duplicates() Will not work ok because if ↵
      ↪ some other column is different they won't be cleared
```

```
[20]: #instead do this :
      df_apps_clean = df_apps_clean.drop_duplicates(subset = ['App', 'Type', 'Price'])
```

```
[21]: duplicates_instagram = df_apps_clean[df_apps_clean['App'] == 'Instagram']
```

```
[22]: duplicates_instagram
```

```
[22]:
```

	App	Category	Rating	Reviews	Size_MBs	Installs	Type	\
10806	Instagram	SOCIAL	4.50	66577313	5.30	1,000,000,000	Free	

	Price	Content_Rating	Genres
10806	0	Teen	Social

```
[23]: df_apps_clean.shape #Itan (9367,10)
```

```
[23]: (8199, 10)
```

7 Find Highest Rated Apps

Challenge: Identify which apps are the highest rated. What problem might you encounter if you rely exclusively on ratings alone to determine the quality of an app?

```
[24]: apps_5_stars = df_apps_clean[df_apps_clean['Rating'] == 5]
```

```
[25]: apps_5_stars
```

```
[25]:
```

	App	Category	Rating	\
21	KBA-EZ Health Guide	MEDICAL	5.00	
28	Ra Ga Ba	GAME	5.00	
47	Mu.F.O.	GAME	5.00	
82	Brick Breaker BR	GAME	5.00	
99	Anatomy & Physiology Vocabulary Exam Review App	MEDICAL	5.00	
...	...	...	...	
2680	Florida Wildflowers	FAMILY	5.00	
2750	Superheroes, Marvel, DC, Comics, TV, Movies News	COMICS	5.00	
3030	CL Keyboard - Myanmar Keyboard (No Ads)	TOOLS	5.00	
3115	Oración CX	LIFESTYLE	5.00	

4058				Ek Bander Ne Kholi Dukan		FAMILY	5.00
------	--	--	--	--------------------------	--	--------	------

	Reviews	Size_MBs	Installs	Type	Price	Content_Rating	Genres
21	4	25.00	1	Free	0	Everyone	Medical
28	2	20.00	1	Paid	\$1.49	Everyone	Arcade
47	2	16.00	1	Paid	\$0.99	Everyone	Arcade
82	7	19.00	5	Free	0	Everyone	Arcade
99	1	4.60	5	Free	0	Everyone	Medical
...	...	...	...	...	...	...	...
2680	5	69.00	1,000	Free	0	Everyone	Education
2750	34	12.00	5,000	Free	0	Everyone	Comics
3030	24	3.20	5,000	Free	0	Everyone	Tools
3115	103	3.80	5,000	Free	0	Everyone	Lifestyle
4058	10	3.00	10,000	Free	0	Everyone	Entertainment

[271 rows x 10 columns]

```
[26]: apps_5_stars.sort_values('Reviews', ascending = False) #All 5 star reviews
      ↪ apps have a very small ammount of votes
```

```
[26]:
```

	App	Category	Rating \
2095	Ríos de Fe	LIFESTYLE	5.00
2438	FD Calculator (EMI, SIP, RD & Loan Eligibility)	FINANCE	5.00
3115	Oración CX	LIFESTYLE	5.00
2107	Barisal University App-BU Face	FAMILY	5.00
2069	Master E.K	FAMILY	5.00
...	...	...	...
1272	CG Prints	PHOTOGRAPHY	5.00
453	Wowkwis aq Ka'qaquj	FAMILY	5.00
349	DF Glue Board	PARENTING	5.00
230	NCLEX Multi-topic Nursing Exam Review-Quiz & n...	MEDICAL	5.00
411	EC SPORTS	SPORTS	5.00

	Reviews	Size_MBs	Installs	Type	Price	Content_Rating \
2095	141	15.00	1,000	Free	0	Everyone
2438	104	2.30	1,000	Free	0	Everyone
3115	103	3.80	5,000	Free	0	Everyone
2107	100	10.00	1,000	Free	0	Everyone
2069	90	19.00	1,000	Free	0	Everyone
...	...	...	...	...	...	...
1272	1	2.30	100	Free	0	Everyone
453	1	49.00	10	Free	0	Everyone
349	1	27.00	10	Free	0	Everyone
230	1	4.20	10	Free	0	Everyone
411	1	6.30	10	Free	0	Everyone

Genres

```

2095      Lifestyle
2438      Finance
3115      Lifestyle
2107      Education
2069      Education
...
1272      Photography
453      Education;Education
349      Parenting
230      Medical
411      Sports

```

```
[271 rows x 10 columns]
```

8 Find 5 Largest Apps in terms of Size (MBs)

Challenge: What's the size in megabytes (MB) of the largest Android apps in the Google Play Store. Based on the data, do you think there could be limit in place or can developers make apps as large as they please?

```
[27]: df_apps_clean.sort_values('Size_MB', ascending = False)
```

```
[27]:
```

	App	Category	Rating	Reviews \
9942	Talking Babsy Baby: Baby Games	LIFESTYLE	4.00	140995
10687	Hungry Shark Evolution	GAME	4.50	6074334
9943	Miami crime simulator	GAME	4.00	254518
9944	Gangster Town: Vice District	FAMILY	4.30	65146
3144	Vi Trainer	HEALTH_AND_FITNESS	3.60	124
...	...	...	...	...
2648	Ad Remove Plugin for App2SD	PRODUCTIVITY	4.10	66
5798	ExDialer PRO Key	COMMUNICATION	4.50	5474
2684	My baby firework (Remove ad)	FAMILY	4.10	30
7966	Market Update Helper	LIBRARIES_AND_DEMO	4.10	20145
4696	Essential Resources	LIBRARIES_AND_DEMO	4.60	237

	Size_MB	Installs	Type	Price	Content_Rating \
9942	100.00	10,000,000	Free	0	Everyone
10687	100.00	100,000,000	Free	0	Teen
9943	100.00	10,000,000	Free	0	Mature 17+
9944	100.00	10,000,000	Free	0	Mature 17+
3144	100.00	5,000	Free	0	Everyone
...	...	...	...	...	...
2648	0.02	1,000	Paid	\$1.29	Everyone
5798	0.02	100,000	Paid	\$3.99	Everyone
2684	0.01	1,000	Paid	\$0.99	Everyone
7966	0.01	1,000,000	Free	0	Everyone
4696	0.01	50,000	Free	0	Everyone

```

          Genres
9942  Lifestyle;Pretend Play
10687          Arcade
9943          Action
9944          Simulation
3144    Health & Fitness
...
2648    Productivity
5798    Communication
2684    Entertainment
7966    Libraries & Demo
4696    Libraries & Demo

```

[8199 rows x 10 columns]

9 Find the 5 App with Most Reviews

Challenge: Which apps have the highest number of reviews? Are there any paid apps among the top 50?

```
[28]: df_apps_clean.sort_values('Reviews', ascending = False)
```

```
[28]:
```

	App	Category \
10805	Facebook	SOCIAL
10785	WhatsApp Messenger	COMMUNICATION
10806	Instagram	SOCIAL
10784	Messenger - Text and Video Chat for Free	COMMUNICATION
10650	Clash of Clans	GAME
...	...	...
453	Wowkwis aq Ka'qaquj	FAMILY
462	CB Fit	HEALTH_AND_FITNESS
901	ES Billing System (Offline App)	PRODUCTIVITY
1416	Ek Kahani Aisi Bhi Season 3 - The Horror Story	FAMILY
1100	Chenoweth AH	MEDICAL

	Rating	Reviews	Size_MBs	Installs	Type	Price	Content_Rating \
10805	4.10	78158306	5.30	1,000,000,000	Free	0	Teen
10785	4.40	69119316	3.50	1,000,000,000	Free	0	Everyone
10806	4.50	66577313	5.30	1,000,000,000	Free	0	Teen
10784	4.00	56642847	3.50	1,000,000,000	Free	0	Everyone
10650	4.60	44891723	98.00	100,000,000	Free	0	Everyone 10+
...	...	...	...	...	...	...	...
453	5.00	1	49.00	10	Free	0	Everyone
462	5.00	1	7.80	10	Free	0	Everyone
901	5.00	1	4.20	100	Free	0	Everyone
1416	3.00	1	5.80	100	Free	0	Teen

1100	5.00	1	27.00	100	Free	0	Everyone
------	------	---	-------	-----	------	---	----------

	Genres
10805	Social
10785	Communication
10806	Social
10784	Communication
10650	Strategy
...	...
453	Education;Education
462	Health & Fitness
901	Productivity
1416	Entertainment
1100	Medical

[8199 rows x 10 columns]

10 Plotly Pie and Donut Charts - Visualise Categorical Data: Content Ratings

```
[29]: ratings = df_apps_clean.Content_Rating.value_counts()
```

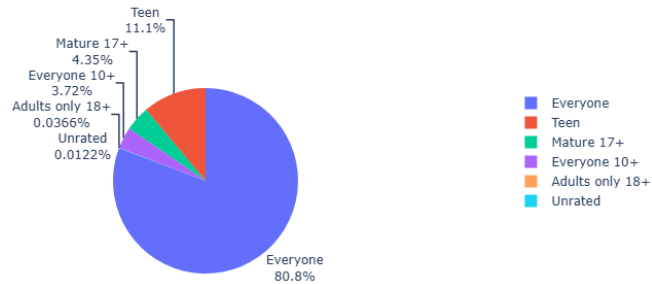
```
[30]: ratings
```

```
[30]: Content_Rating
Everyone          6621
Teen              912
Mature 17+       357
Everyone 10+     305
Adults only 18+   3
Unrated          1
Name: count, dtype: int64
```

```
[31]: import plotly.express as px
```

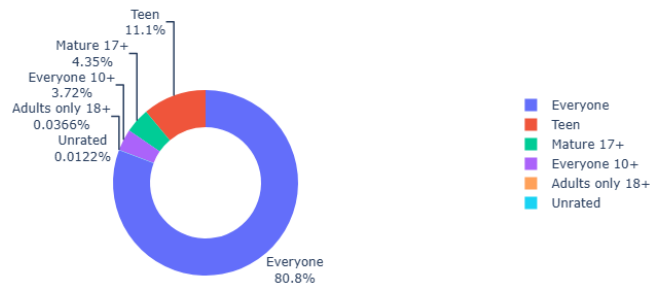
```
[32]: piechart = px.pie(labels = ratings.index, values = ratings.values, title = 'Content Rating', names = ratings.index,)
piechart.update_traces(textposition = 'outside', textinfo='percent+label')
piechart.show()
```

Content Rating



```
[33]: #add the parameter hole = (0 to 1) to create a donut chart
piechart = px.pie(labels = ratings.index, values = ratings.values, title = 'Content Rating', names = ratings.index, hole = 0.6)
piechart.update_traces(textposition = 'outside', textinfo='percent+label')
piechart.show()
```

Content Rating



11 Numeric Type Conversion: Examine the Number of Installs

Challenge: How many apps had over 1 billion (that's right - BILLION) installations? How many apps just had a single install?

Check the datatype of the Installs column.

Count the number of apps at each level of installations.

Convert the number of installations (the Installs column) to a numeric data type. Hint: this is a 2-step process. You'll have to make sure you remove non-numeric characters first.

```
[34]: df_apps_clean.Installs.describe()
```

```
[34]: count          8199
      unique          19
      top        1,000,000
      freq          1417
      Name: Installs, dtype: object
```

```
[35]: df_apps_clean.info() #enallaktika
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8199 entries, 21 to 10835
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   App              8199 non-null   object
1   Category         8199 non-null   object
2   Rating           8199 non-null   float64
3   Reviews          8199 non-null   int64
4   Size_MBs         8199 non-null   float64
5   Installs         8199 non-null   object
6   Type             8199 non-null   object
7   Price            8199 non-null   object
8   Content_Rating   8199 non-null   object
9   Genres           8199 non-null   object
dtypes: float64(2), int64(1), object(7)
memory usage: 704.6+ KB
```

```
[36]: df_apps_clean[['App', 'Installs']].groupby('Installs').count() #de mas_
      ↪ aresoun ta komma
```

```
[36]:
```

	App
Installs	
1	3
1,000	698
1,000,000	1417
1,000,000,000	20
10	69
10,000	988
10,000,000	933
100	303
100,000	1096
100,000,000	189
5	9
5,000	425
5,000,000	607
50	56
50,000	457
50,000,000	202
500	199

500,000	504
500,000,000	24

```
[37]: df_apps_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8199 entries, 21 to 10835
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   App              8199 non-null   object
1   Category         8199 non-null   object
2   Rating           8199 non-null   float64
3   Reviews          8199 non-null   int64
4   Size_MBs         8199 non-null   float64
5   Installs         8199 non-null   object
6   Type             8199 non-null   object
7   Price            8199 non-null   object
8   Content_Rating   8199 non-null   object
9   Genres           8199 non-null   object
dtypes: float64(2), int64(1), object(7)
memory usage: 704.6+ KB
```

```
[38]: df_apps_clean.Installs = df_apps_clean.Installs.astype(str).str.replace(',', '')
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_8840\70812682.py:1:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[39]: df_apps_clean.Installs = pd.to_numeric(df_apps_clean.Installs)
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_8840\665362016.py:1:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[40]: df_apps_clean[['App', 'Installs']].groupby('Installs').count()
```

```
[40]:
```

	App
Installs	
1	3
5	9
10	69
50	56
100	303
500	199
1000	698
5000	425
10000	988
50000	457
100000	1096
500000	504
1000000	1417
5000000	607
10000000	933
50000000	202
100000000	189
500000000	24
1000000000	20

12 Find the Most Expensive Apps, Filter out the Junk, and Calculate a (ballpark) Sales Revenue Estimate

Let's examine the Price column more closely.

Challenge: Convert the price column to numeric data. Then investigate the top 20 most expensive apps in the dataset.

Remove all apps that cost more than \$250 from the `df_apps_clean` DataFrame.

Add a column called 'Revenue_Estimate' to the DataFrame. This column should hold the price of the app times the number of installs. What are the top 10 highest grossing paid apps according to this estimate? Out of the top 10 highest grossing paid apps, how many are games?

```
[41]: df_apps_clean['Price'].describe()
```

```
[41]: count      8199
      unique       73
      top         0
      freq      7595
      Name: Price, dtype: object
```

```
[42]: df_apps_clean.Price = df_apps_clean.Price.astype(str).str.replace('$', "")
      df_apps_clean.Price = pd.to_numeric(df_apps_clean.Price)
```



```
df_apps_clean.sort_values('Price', ascending = False).head(20)
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_8840\3837063921.py:1:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
C:\Users\user\AppData\Local\Temp\ipykernel_8840\3837063921.py:2:
SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[42]:
```

	App	Category	Rating	Reviews	Size_MBs	\
3946	I'm Rich - Trump Edition	LIFESTYLE	3.60	275	7.30	
2461	I AM RICH PRO PLUS	FINANCE	4.00	36	41.00	
4606	I Am Rich Premium	FINANCE	4.10	1867	4.70	
3145	I am rich(premium)	FINANCE	3.50	472	0.94	
3554	I'm rich	LIFESTYLE	3.80	718	26.00	
5765	I am rich	LIFESTYLE	3.80	3547	1.80	
1946	I am rich (Most expensive app)	FINANCE	4.10	129	2.70	
2775	I Am Rich Pro	FAMILY	4.40	201	2.70	
3221	I am Rich Plus	FAMILY	4.00	856	8.70	
3114	I am Rich	FINANCE	4.30	180	3.80	
1331	most expensive app (H)	FAMILY	4.30	6	1.50	
2394	I am Rich!	FINANCE	3.80	93	22.00	
3897	I Am Rich	FAMILY	3.60	217	4.90	
2193	I am extremely Rich	LIFESTYLE	2.90	41	2.90	
3856	I am rich VIP	LIFESTYLE	3.80	411	2.60	
2281	Vargo Anesthesia Mega App	MEDICAL	4.60	92	32.00	
1407	LTC AS Legal	MEDICAL	4.00	6	1.30	
2629	I am Rich Person	LIFESTYLE	4.20	134	1.80	
2481	A Manual of Acupuncture	MEDICAL	3.50	214	68.00	
4264	Golfshot Plus: Golf GPS	SPORTS	4.10	3387	25.00	

	Installs	Type	Price	Content_Rating	Genres
3946	10000	Paid	400.00	Everyone	Lifestyle

2461	1000	Paid	399.99	Everyone	Finance
4606	50000	Paid	399.99	Everyone	Finance
3145	5000	Paid	399.99	Everyone	Finance
3554	10000	Paid	399.99	Everyone	Lifestyle
5765	100000	Paid	399.99	Everyone	Lifestyle
1946	1000	Paid	399.99	Teen	Finance
2775	5000	Paid	399.99	Everyone	Entertainment
3221	10000	Paid	399.99	Everyone	Entertainment
3114	5000	Paid	399.99	Everyone	Finance
1331	100	Paid	399.99	Everyone	Entertainment
2394	1000	Paid	399.99	Everyone	Finance
3897	10000	Paid	389.99	Everyone	Entertainment
2193	1000	Paid	379.99	Everyone	Lifestyle
3856	10000	Paid	299.99	Everyone	Lifestyle
2281	1000	Paid	79.99	Everyone	Medical
1407	100	Paid	39.99	Everyone	Medical
2629	1000	Paid	37.99	Everyone	Lifestyle
2481	1000	Paid	33.99	Everyone	Medical
4264	50000	Paid	29.99	Everyone	Sports

12.0.1 The most expensive apps sub \$250

```
[43]: df_apps_clean = df_apps_clean[df_apps_clean['Price'] < 250]
df_apps_clean.sort_values('Price', ascending=False).head(5)
```

```
[43]:
```

	App	Category	Rating	Reviews	Size_MBs	\
2281	Vargo Anesthesia Mega App	MEDICAL	4.60	92	32.00	
1407	LTC AS Legal	MEDICAL	4.00	6	1.30	
2629	I am Rich Person	LIFESTYLE	4.20	134	1.80	
2481	A Manual of Acupuncture	MEDICAL	3.50	214	68.00	
2463	PTA Content Master	MEDICAL	4.20	64	41.00	

	Installs	Type	Price	Content_Rating	Genres
2281	1000	Paid	79.99	Everyone	Medical
1407	100	Paid	39.99	Everyone	Medical
2629	1000	Paid	37.99	Everyone	Lifestyle
2481	1000	Paid	33.99	Everyone	Medical
2463	1000	Paid	29.99	Everyone	Medical

12.0.2 Highest Grossing Paid Apps (ballpark estimate)

```
[44]: df_apps_clean['Revenue Estimate'] = df_apps_clean.Installs.mul(df_apps_clean.
    ↪Price)
df_apps_clean.sort_values('Revenue Estimate', ascending = False)[:10]
```

```
[44]:
```

	App	Category	Rating	Reviews	Size_MBs	\
9220	Minecraft	FAMILY	4.50	2376564	19.00	

8825	Hitman Sniper	GAME	4.60	408292	29.00
7151	Grand Theft Auto: San Andreas	GAME	4.40	348962	26.00
7477	Facetune - For Free	PHOTOGRAPHY	4.40	49553	48.00
7977	Sleep as Android Unlock	LIFESTYLE	4.50	23966	0.85
6594	DraStic DS Emulator	GAME	4.60	87766	12.00
6082	Weather Live	WEATHER	4.50	76593	4.75
7954	Bloons TD 5	FAMILY	4.60	190086	94.00
7633	Five Nights at Freddy's	GAME	4.60	100805	50.00
6746	Card Wars - Adventure Time	FAMILY	4.30	129603	23.00

	Installs	Type	Price	Content_Rating	Genres \
9220	10000000	Paid	6.99	Everyone 10+	Arcade;Action & Adventure
8825	10000000	Paid	0.99	Mature 17+	Action
7151	1000000	Paid	6.99	Mature 17+	Action
7477	1000000	Paid	5.99	Everyone	Photography
7977	1000000	Paid	5.99	Everyone	Lifestyle
6594	1000000	Paid	4.99	Everyone	Action
6082	500000	Paid	5.99	Everyone	Weather
7954	1000000	Paid	2.99	Everyone	Strategy
7633	1000000	Paid	2.99	Teen	Action
6746	1000000	Paid	2.99	Everyone 10+	Card;Action & Adventure

	Revenue Estimate
9220	69,900,000.00
8825	9,900,000.00
7151	6,990,000.00
7477	5,990,000.00
7977	5,990,000.00
6594	4,990,000.00
6082	2,995,000.00
7954	2,990,000.00
7633	2,990,000.00
6746	2,990,000.00

13 Plotly Bar Charts & Scatter Plots: Analysing App Categories

```
[45]: df_apps_clean.Category.nunique()
```

```
[45]: 33
```

```
[46]: df_apps_clean.Category.value_counts()
```

```
[46]: Category
FAMILY      1606
GAME         910
TOOLS        719
```

PRODUCTIVITY	301
PERSONALIZATION	298
LIFESTYLE	297
FINANCE	296
MEDICAL	292
PHOTOGRAPHY	263
BUSINESS	262
SPORTS	260
COMMUNICATION	257
HEALTH_AND_FITNESS	243
NEWS_AND_MAGAZINES	204
SOCIAL	203
TRAVEL_AND_LOCAL	187
SHOPPING	180
BOOKS_AND_REFERENCE	169
VIDEO_PLAYERS	148
DATING	134
EDUCATION	118
MAPS_AND_NAVIGATION	118
ENTERTAINMENT	102
FOOD_AND_DRINK	94
AUTO_AND_VEHICLES	73
WEATHER	72
LIBRARIES_AND_DEMO	64
HOUSE_AND_HOME	62
ART_AND_DESIGN	61
COMICS	54
PARENTING	50
EVENTS	45
BEAUTY	42

Name: count, dtype: int64

```
[47]: top10_category = df_apps_clean.Category.value_counts()[:10]
```

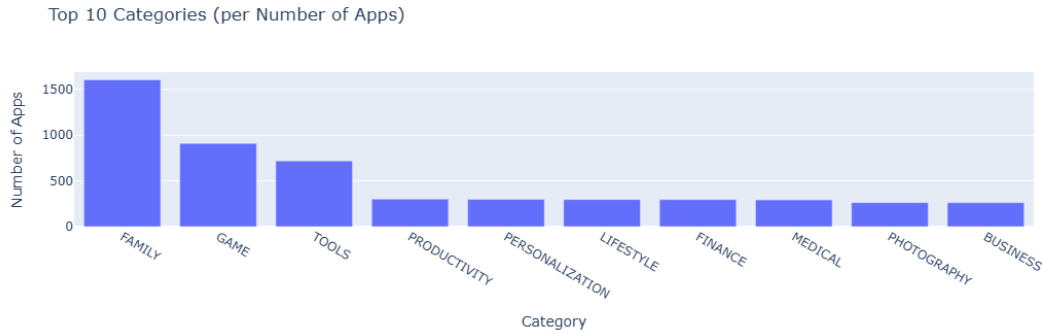
```
[48]: top10_category
```

```
[48]: Category
FAMILY          1606
GAME             910
TOOLS            719
PRODUCTIVITY    301
PERSONALIZATION 298
LIFESTYLE       297
FINANCE         296
MEDICAL         292
PHOTOGRAPHY     263
BUSINESS        262
```

Name: count, dtype: int64

13.0.1 Vertical Bar Chart - Highest Competition (Number of Apps)

```
[49]: bar = px.bar(x = top10_category.index, y = top10_category.values, title = 'Top 10 Categories (per Number of Apps)')
bar.update_layout(xaxis_title = 'Category', yaxis_title = 'Number of Apps')
bar.show()
```

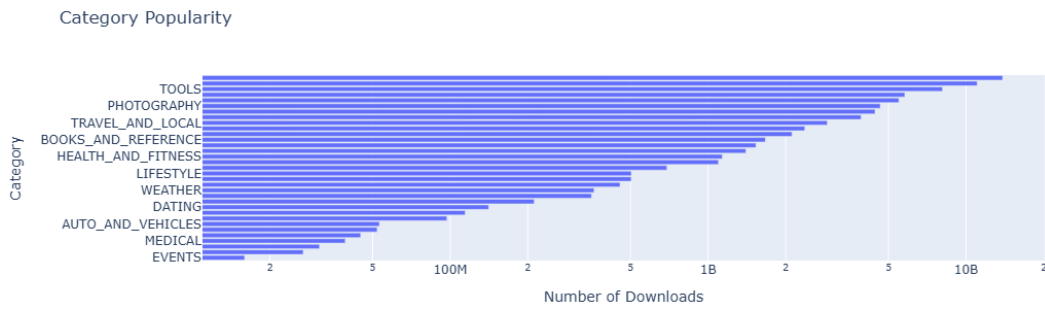


13.0.2 Horizontal Bar Chart - Most Popular Categories (Highest Downloads)

```
[50]: category_installs = df_apps_clean.groupby('Category').agg({'Installs':pd.Series.
    ↳sum})
category_installs.sort_values('Installs', ascending = True, inplace = True)
category_installs.head(5)
```

```
[50]:      Installs
Category
EVENTS    15949410
BEAUTY    26916200
PARENTING 31116110
MEDICAL   39162676
COMICS    44931100
```

```
[74]: h_bar = px.bar(x = category_installs.Installs, y = category_installs.index,
    ↳orientation='h', title = 'Category Popularity')
h_bar.update_layout(xaxis_title = 'Number of Downloads', yaxis_title =
    ↳'Category', xaxis = dict(type='log'))
h_bar.show()
```



13.0.3 Category Concentration - Downloads vs. Competition

Challenge: * First, create a DataFrame that has the number of apps in one column and the number of installs in another:

- Then use the [plotly express examples from the documentation](#) alongside the `.scatter()` API [reference](#) to create scatter plot that looks like this.

Hint: Use the `size`, `hover_name` and `color` parameters in `.scatter()`. To scale the yaxis, call `.update_layout()` and specify that the yaxis should be on a log-scale like so: `yaxis=dict(type='log')`

```
[52]: cat_number = df_apps_clean.groupby('Category').agg({'App':pd.Series.count})
```

```
[53]: cat_number.head(5)
```

```
[53]:
```

Category	App
ART_AND_DESIGN	61
AUTO_AND_VEHICLES	73
BEAUTY	42
BOOKS_AND_REFERENCE	169
BUSINESS	262

```
[54]: cat_merged_df = pd.merge(cat_number, category_installs, on = 'Category', how = 'inner')
```

```
[55]: print(f'The dimensions of the DataFrame are: {cat_merged_df.shape}')
```

The dimensions of the DataFrame are: (33, 2)

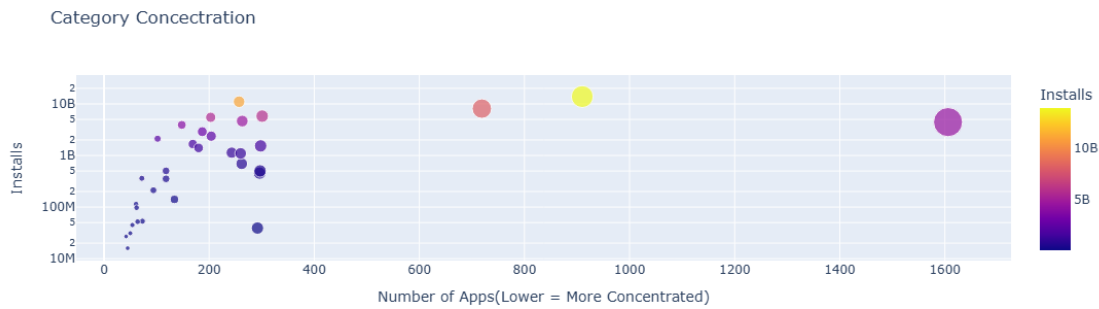
```
[56]: cat_merged_df.sort_values('Installs', ascending=False)
```

```
[56]:
```

Category	App	Installs
GAME	910	13858762717
COMMUNICATION	257	11039241530

TOOLS	719	8099724500
PRODUCTIVITY	301	5788070180
SOCIAL	203	5487841475
PHOTOGRAPHY	263	4649143130
FAMILY	1606	4437554490
VIDEO_PLAYERS	148	3916897200
TRAVEL_AND_LOCAL	187	2894859300
NEWS_AND_MAGAZINES	204	2369110650
ENTERTAINMENT	102	2113660000
BOOKS_AND_REFERENCE	169	1665791655
PERSONALIZATION	298	1532352930
SHOPPING	180	1400331540
HEALTH_AND_FITNESS	243	1134006220
SPORTS	260	1096431465
BUSINESS	262	692018120
LIFESTYLE	297	503611120
MAPS_AND_NAVIGATION	118	503267560
FINANCE	296	455249400
WEATHER	72	361096500
EDUCATION	118	352852000
FOOD_AND_DRINK	94	211677750
DATING	134	140912410
ART_AND_DESIGN	61	114233100
HOUSE_AND_HOME	62	97082000
AUTO_AND_VEHICLES	73	53129800
LIBRARIES_AND_DEMO	64	52083000
COMICS	54	44931100
MEDICAL	292	39162676
PARENTING	50	31116110
BEAUTY	42	26916200
EVENTS	45	15949410

```
[57]: scatter = px.scatter(cat_merged_df,
                           x = 'App',
                           y = 'Installs',
                           title = 'Category Concestration',
                           size = 'App',
                           hover_name = cat_merged_df.index,
                           color = 'Installs')
scatter.update_layout(xaxis_title = "Number of Apps(Lower = More Concentrated)",
                      yaxis_title = "Installs",
                      yaxis = dict(type='log'))
scatter.show()
```



14 Extracting Nested Data from a Column

Challenge: How many different types of genres are there? Can an app belong to more than one genre? Check what happens when you use `.value_counts()` on a column with nested values? See if you can work around this problem by using the `.split()` function and the DataFrame's `.stack()` method.

```
[114]: df_apps_clean.Genres.describe()
# print(f"There are {df_apps_clean.Genres.nunique} different genres")
df_apps_clean.Genres.value_counts()
```

```
[114]: Genres
Tools                                718
Entertainment                        467
Education                           429
Productivity                         301
Personalization                      298
...
Adventure;Brain Games                1
Travel & Local;Action & Adventure    1
Art & Design;Pretend Play            1
Music & Audio;Music & Video          1
Lifestyle;Pretend Play               1
Name: count, Length: 114, dtype: int64
```

```
[116]: genres = len(df_apps_clean.Genres.value_counts())
print(f"There are {genres} different genres or combinations of genres. I want_
↳to find the number of the unique single genres")
```

There are 114 different genres or combinations of genres. I want to find the number of the unique single genres

14.0.1 We need to separate the multiple genres at the colon separator.

```
[118]: stack = df_apps_clean.Genres.str.split(':', expand = True).stack()  
print(f"We now have a single column with shape: {stack.shape}")
```

We now have a single column with shape: (8564,)

```
[120]: num_genres = stack.value_counts()  
num_genres
```

```
[120]: Tools                719  
       Education           587  
       Entertainment       498  
       Action              304  
       Productivity        301  
       Personalization     298  
       Lifestyle           298  
       Finance             296  
       Medical             292  
       Sports              270  
       Photography         263  
       Business            262  
       Communication       258  
       Health & Fitness    245  
       Casual              216  
       News & Magazines    204  
       Social              203  
       Simulation          200  
       Travel & Local      187  
       Arcade              185  
       Shopping            180  
       Books & Reference   171  
       Video Players & Editors 150  
       Dating              134  
       Puzzle              124  
       Maps & Navigation   118  
       Role Playing        111  
       Racing              103  
       Action & Adventure   96  
       Strategy            95  
       Food & Drink         94  
       Educational         93  
       Adventure           78  
       Auto & Vehicles      73  
       Weather             72  
       Pretend Play        68  
       Brain Games         65  
       Libraries & Demo     64
```

Art & Design	62
House & Home	62
Board	57
Comics	54
Parenting	50
Card	46
Events	45
Beauty	42
Casino	37
Music & Video	31
Creativity	31
Trivia	28
Word	22
Music	21
Music & Audio	1

Name: count, dtype: int64

```
[125]: print(f"Number of genres : {len(num_genres)}")
```

Number of genres : 53

15 Colour Scales in Plotly Charts - Competition in Genres

Challenge: Can you create this chart with the Series containing the genre data?

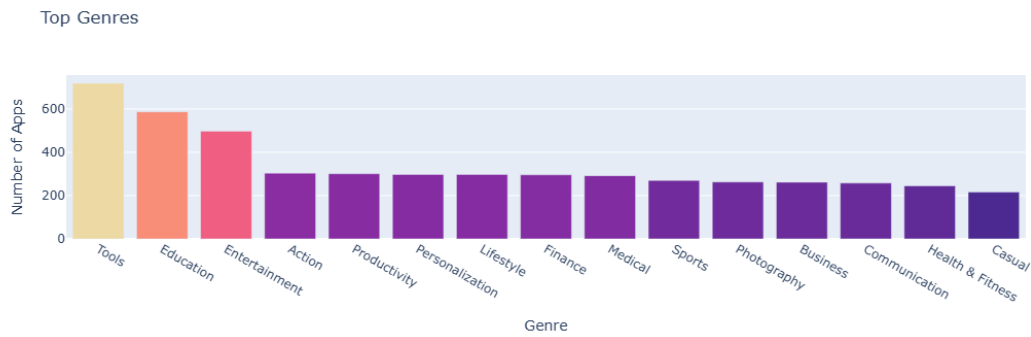
Try experimenting with the built in colour scales in Plotly. You can find a full list [here](#).

- Find a way to set the colour scale using the `color_continuous_scale` parameter.
- Find a way to make the color axis disappear by using `coloraxis_showscale`.

```
[131]: top_15_genres = num_genres[:15]
bar = px.bar(x = top_15_genres.index,
             y = top_15_genres.values,
             title = 'Top Genres',
             color = top_15_genres.values,
             hover_name = top_15_genres.index,
             color_continuous_scale='Agunset')

bar.update_layout(xaxis_title = 'Genre', yaxis_title = 'Number of Apps',
                  coloraxis_showscale=False)

bar.show()
```



16 Grouped Bar Charts: Free vs. Paid Apps per Category

```
[132]: df_apps_clean.Type.value_counts()
```

```
[132]: Type
Free    7595
Paid     589
Name: count, dtype: int64
```

17 We see that the majority of apps are free on the Google Play Store. But perhaps some categories have more paid apps than others. Let's investigate. We can group our data first by Category and then by Type. Then we can add up the number of apps per each type. Using `as_index=False` we push all the data into columns rather than end up with our Categories as the index.

```
[144]: df_free_vs_paid = df_apps_clean.groupby(['Category', 'Type'], as_index = False).
        ↪agg({'App' : pd.Series.count})
df_free_vs_paid
```

```
[144]:
```

	Category	Type	App
0	ART_AND_DESIGN	Free	58
1	ART_AND_DESIGN	Paid	3
2	AUTO_AND_VEHICLES	Free	72
3	AUTO_AND_VEHICLES	Paid	1
4	BEAUTY	Free	42
..	...	...	...
56	TRAVEL_AND_LOCAL	Paid	8
57	VIDEO_PLAYERS	Free	144

```

58     VIDEO_PLAYERS  Paid    4
59         WEATHER  Free   65
60         WEATHER  Paid    7

```

[61 rows x 3 columns]

```
[145]: df_free_vs_paid.sort_values('App')
```

```

[145]:
      Category  Type  App
3  AUTO_AND_VEHICLES  Paid    1
24  FOOD_AND_DRINK  Paid    2
38  NEWS_AND_MAGAZINES  Paid    2
40      PARENTING  Paid    2
17  ENTERTAINMENT  Paid    2
..      ...      ...
31      LIFESTYLE  Free  284
21      FINANCE  Free  289
53      TOOLS  Free  656
25      GAME  Free  834
19      FAMILY  Free 1456

```

[61 rows x 3 columns]

Challenge: Use the plotly express bar [chart examples](#) and the `.bar()` [API reference](#) to create this bar chart:

You'll want to use the `df_free_vs_paid` DataFrame that you created above that has the total number of free and paid apps per category.

See if you can figure out how to get the look above by changing the `categoryorder` to 'total descending' as outlined in the documentation [here](#).

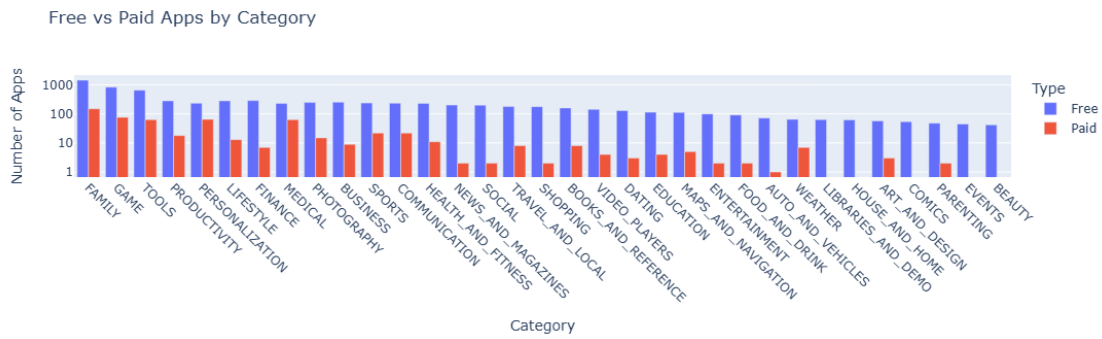
```

[174]: g_bar = px.bar(df_free_vs_paid,
                    x = 'Category',
                    y = 'App',
                    title = 'Free vs Paid Apps by Category',
                    color = 'Type',
                    barmode = 'group')

g_bar.update_layout(xaxis_title = 'Category',
                    yaxis_title = 'Number of Apps',
                    xaxis_tickangle = 45,
                    xaxis = {'categoryorder' : 'total descending'},
                    yaxis = dict(type = 'log'))

g_bar.show()

```



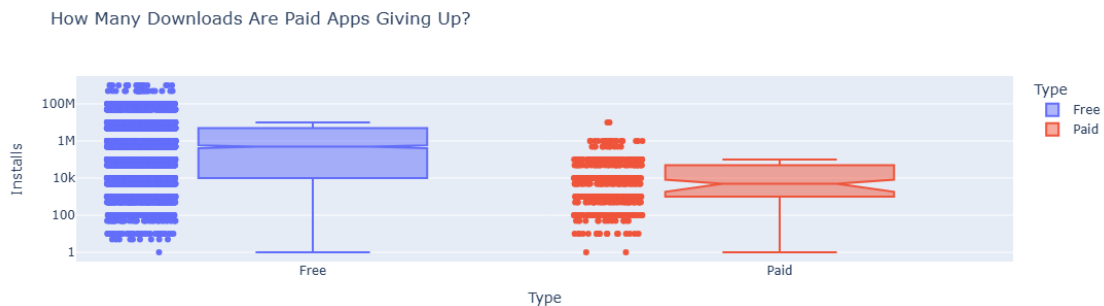
[]:

18 Plotly Box Plots: Lost Downloads for Paid Apps

Challenge: Create a box plot that shows the number of Installs for free versus paid apps. How does the median number of installations compare? Is the difference large or small?

Use the [Box Plots Guide](#) and the [.box API reference](#) to create the following chart.

```
[194]: box = px.box(df_apps_clean,
                  x = 'Type',
                  y = 'Installs',
                  color = 'Type',
                  notched = True,
                  points = 'all',
                  title = 'How Many Downloads Are Paid Apps Giving Up?')
box.update_layout(yaxis = dict(type = 'log'))
box.show()
```



19 Plotly Box Plots: Revenue by App Category

Challenge: See if you can generate the chart below:

Looking at the hover text, how much does the median app earn in the Tools category? If developing an Android app costs \$30,000 or thereabouts, does the average photography app recoup its development costs?

Hint: I've used 'min ascending' to sort the categories.

```
[216]: df_paid_apps = df_apps_clean[df_apps_clean['Type'] == 'Paid']
df_paid_apps
```

```
[216]:
```

	App	Category	Rating \
28	Ra Ga Ba	GAME	5.00
47	Mu.F.O.	GAME	5.00
233	Chess of Blades (BL/Yaoi Game) (No VA)	FAMILY	4.80
248	The DG Buddy	BUSINESS	3.70
291	AC DC Power Monitor	LIFESTYLE	5.00
...	...	...	...
7957	League of Stickman 2018- Ninja Arena PVP(Dream...	GAME	4.40
7977	Sleep as Android Unlock	LIFESTYLE	4.50
7988	Where's My Water?	FAMILY	4.70
8825	Hitman Sniper	GAME	4.60
9220	Minecraft	FAMILY	4.50

	Reviews	Size_MBs	Installs	Type	Price	Content_Rating \
28	2	20.00	1	Paid	1.49	Everyone
47	2	16.00	1	Paid	0.99	Everyone
233	4	23.00	10	Paid	14.99	Teen
248	3	11.00	10	Paid	2.49	Everyone
291	1	1.20	10	Paid	3.04	Everyone
...	...	...	...	...	...	...
7957	32496	99.00	1000000	Paid	0.99	Teen
7977	23966	0.85	1000000	Paid	5.99	Everyone
7988	188740	69.00	1000000	Paid	1.99	Everyone
8825	408292	29.00	10000000	Paid	0.99	Mature 17+
9220	2376564	19.00	10000000	Paid	6.99	Everyone 10+

	Genres	Revenue Estimate
28	Arcade	1.49
47	Arcade	0.99
233	Casual	149.90
248	Business	24.90
291	Lifestyle	30.40
...	...	...
7957	Action	990,000.00
7977	Lifestyle	5,990,000.00
7988	Puzzle;Brain Games	1,990,000.00

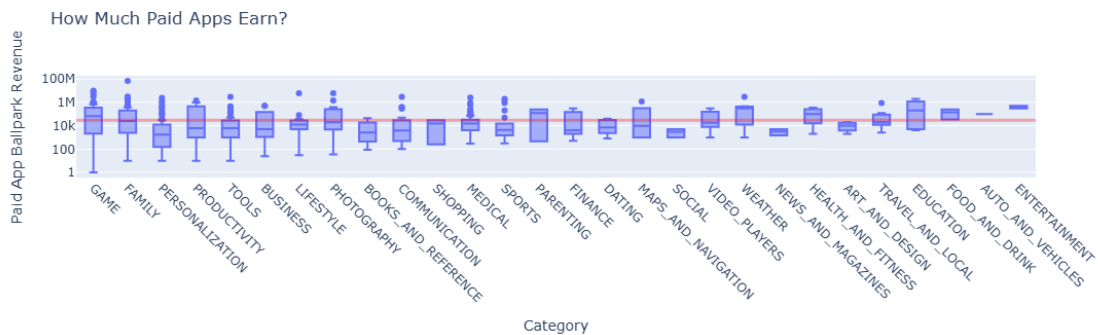
8825	Action	9,900,000.00
9220	Arcade;Action & Adventure	69,900,000.00

[589 rows x 11 columns]

```
[229]: box = px.box(df_paid_apps,
                  x = 'Category',
                  y = 'Revenue Estimate',
                  title = 'How Much Paid Apps Earn?')

box.update_layout(xaxis_title = 'Category',
                  yaxis_title = 'Paid App Ballpark Revenue',
                  xaxis_tickangle = 45,
                  xaxis = {'categoryorder' : 'min ascending'},
                  yaxis = dict(type = 'log'))
box.add_hline(y=30000, line_color='red', line_width=3, opacity = 0.3)

box.show()
```



20 How Much Can You Charge? Examine Paid App Pricing Strategies by Category

Challenge: What is the median price for a paid app? Then compare pricing by category by creating another box plot. But this time examine the prices (instead of the revenue estimates) of the paid apps. I recommend using `{categoryorder': 'max descending'}` to sort the categories.

```
[222]: df_paid_apps.Price.median()
```

[222]: 2.99

```
[230]: box = px.box(df_paid_apps,
                  x='Category',
                  y="Price",
```

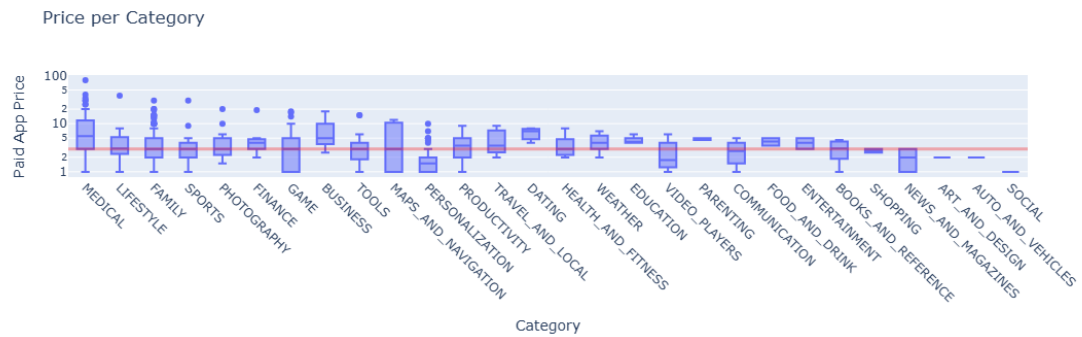
```

        title='Price per Category')

box.update_layout(xaxis_title='Category',
                  yaxis_title='Paid App Price',
                  xaxis={'categoryorder': 'max descending'},
                  xaxis_tickangle = 45,
                  yaxis=dict(type='log'))
box.add_hline(y=2.99 , line_color='red', line_width=3, opacity = 0.3)

box.show()

```



[]: